Applied Security Lab Write-Up

Illicit Consent Grant via an Over-Scoped Third-Party OAuth App

Lab title	Illicit OAuth Consent Grant — Microsoft 365 / Entra ID
Author	Wafula Mwangi
Environment	Microsoft 365 E5 trial tenant — wafulamwangi.onmicrosoft.com (isolated lab tenant, no production data)
Date	October 2026
Target platform	Microsoft Entra ID / Microsoft Graph API
Classification	Internal lab exercise

1. Executive Summary

This lab simulates a real-world illicit consent grant attack against a Microsoft 365 / Entra ID tenant. A deliberately over-scoped, unpublished third-party application ("Expense Reporting Tool") was registered and configured to request broad delegated Microsoft Graph permissions — mail, files, contacts, and persistent offline access. The exercise demonstrates two distinct attack paths, tenant-default protections that blocked the naive path, and a realistic bypass via administrator consent that ultimately granted the malicious app silent, tenant-wide access to every user's mail, files, and contacts.

The key finding is that Microsoft's current secure defaults (blocking direct user consent for non-low-impact scopes) are effective, but are easily undermined the moment an administrator approves an unverified app without scrutinizing the requested scopes. At which point the control is bypassed for every user in the organization simultaneously, with zero further friction or visibility to those users.

2. Scenario

A fictitious SaaS application, "Expense Reporting Tool," was registered within the test tenant to represent a plausible business application an employee or administrator might reasonably encounter and approve. The app requested the following delegated Microsoft Graph permissions, none of which require admin consent by default:

- Mail.Read — Read user mail
- Files.Read.All — Read all files that the user can access
- Contacts.Read — Read user contacts
- offline_access — Maintain access to data the user has granted, via a long-lived refresh token

This permission combination mirrors a common real-world pattern: individually low-risk-looking scopes that, combined with offline_access, grant an attacker persistent, renewable access to a victim's mailbox, file store, and contact list. Long after the initial consent event.

3. Attack Narrative

3.1 Path A — Direct user consent (blocked by tenant defaults)

The first attempt simulated a phishing-style consent-grant attack: a test user (finance.admin@wafulamwangi.onmicrosoft.com) was sent a crafted OAuth authorization URL and asked to sign in and consent directly.

Result: Microsoft Entra ID's current tenant default blocked this outright, returning a "Need admin approval" screen rather than a consent prompt — because the requested scopes were not classified as low-impact.

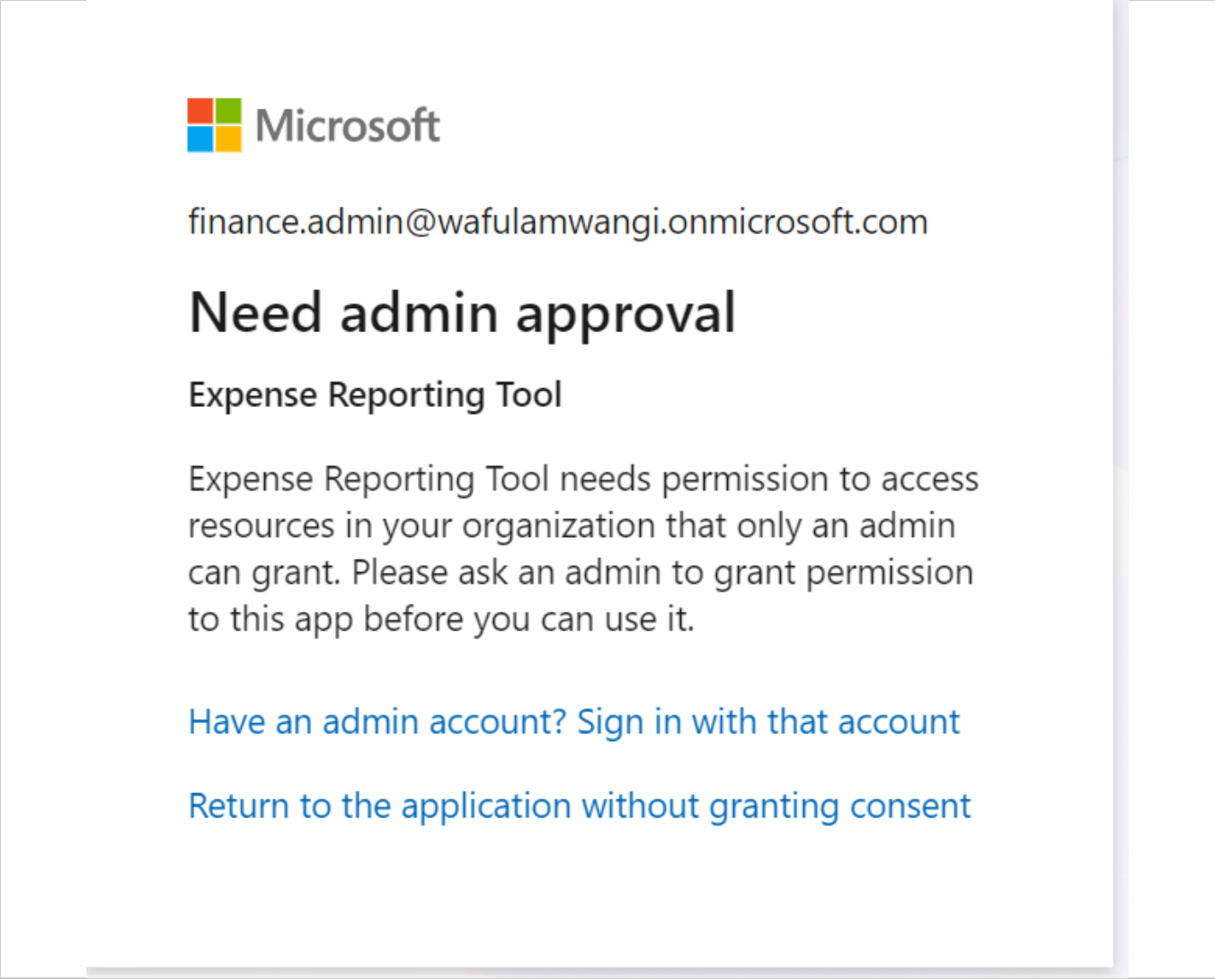


Figure 1 — "Need admin approval" block (Path A, direct user consent denied)

This is a positive finding: out-of-the-box, Microsoft now requires scopes like Mail.Read and offline_access to be explicitly classified as "low impact" before ordinary users can consent to them unassisted. This control meaningfully raises the bar against opportunistic consent-phishing.

3.2 Reconfiguration: Simulating a looser, more common tenant posture

To continue the exercise and reflect tenants where this setting has been loosened (common in organizations that have enabled broader app self-service), the four target permissions were manually added to the tenant's "low impact" permission classification list under Entra ID → Enterprise applications → Consent and permissions.

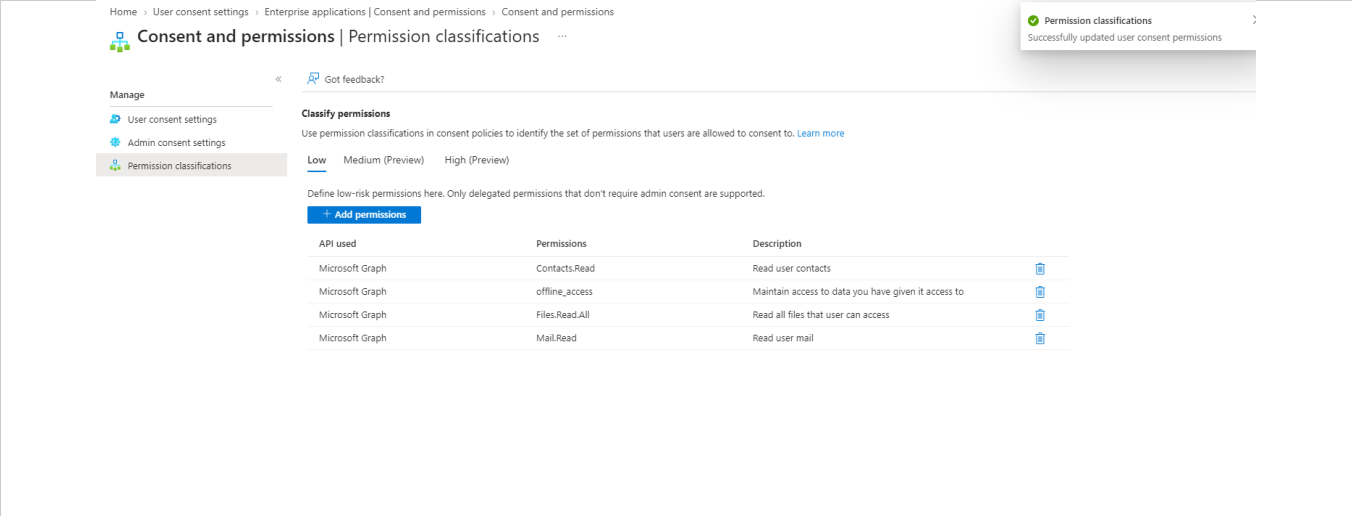


Figure 2 — Permission classifications screen, 4 scopes added to "Low"

Notably, offline_access appears in Microsoft's own suggested "most used permissions" quick-add list for low-risk classification. An administrator moving quickly through this screen could easily low-risk-classify a scope that grants persistent, renewable access without registering its significance.

3.3 Path B: Administrator consent bypass (successful)

Even after loosening the low-impact classification, the direct delegated flow continued to route to admin approval for this particular scope combination. This is consistent with Microsoft's risk-based consent heuristics applying additional scrutiny to high-value scope combinations regardless of individual classification. The attack path was therefore completed via the realistic and arguably more common real-world route: administrator consent.

The tenant Global Administrator was directed to the application's admin consent endpoint and shown the following permissions request:

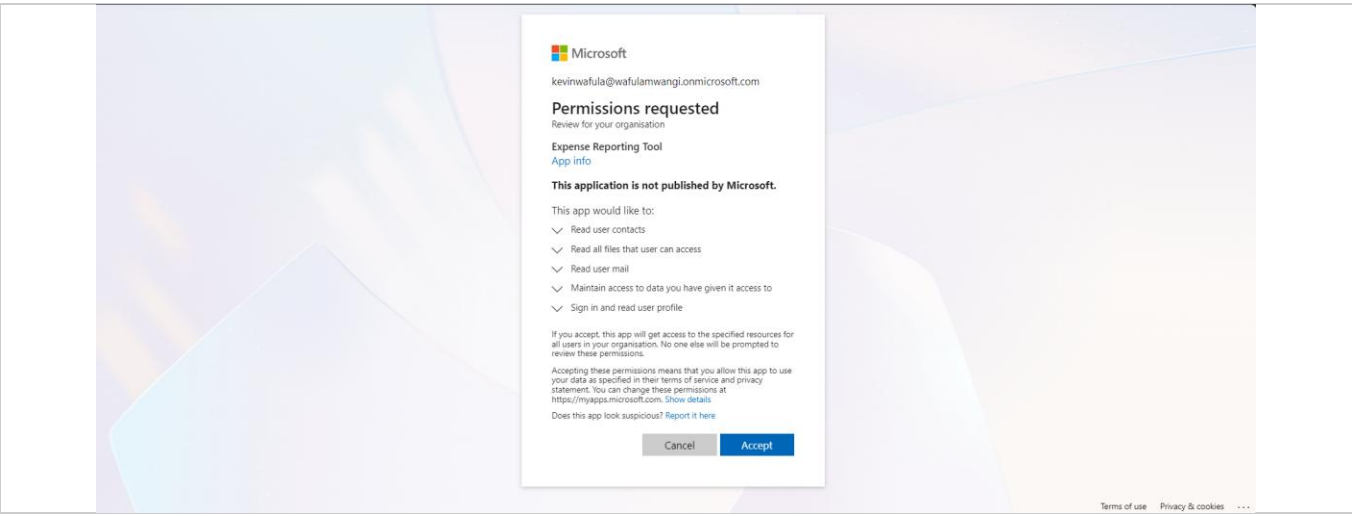


Figure 3 — Admin consent screen: "Permissions requested... for all users in your organisation"

The consent screen explicitly states: "This application is not published by Microsoft" and "If you accept, this app will get access to the specified resources for all users in your organization. No one else will be prompted to review these permissions." Despite this clear warning, the admin account accepted the request — reflecting the realistic failure mode of a time-pressed administrator approving a plausibly named app without close review.

Following acceptance, the Enterprise Application's Permissions blade confirmed tenant-wide grant:

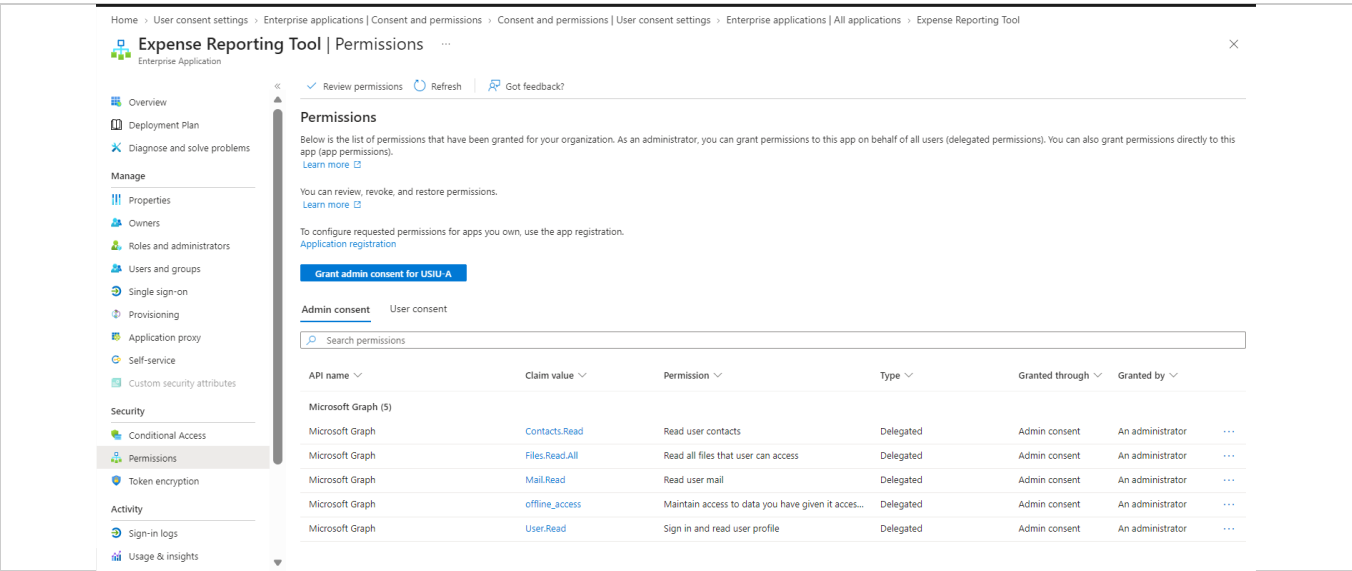


Figure 4 — Enterprise Applications → Permissions, 5 scopes granted via "Admin consent" / "An administrator"

3.4 Silent inheritance by a standard user

With tenant-wide admin consent in place, the Finance Admin test user re-initiated the delegated OAuth authorization code flow. The consent prompt was skipped entirely — the user was redirected straight to the application's callback URI with a valid authorization code, having never seen or approved any permissions screen.

The authorization code was exchanged for an access token via the Microsoft identity platform token endpoint (OAuth2 authorization code grant). The resulting token's claims were inspected via jwt.ms:

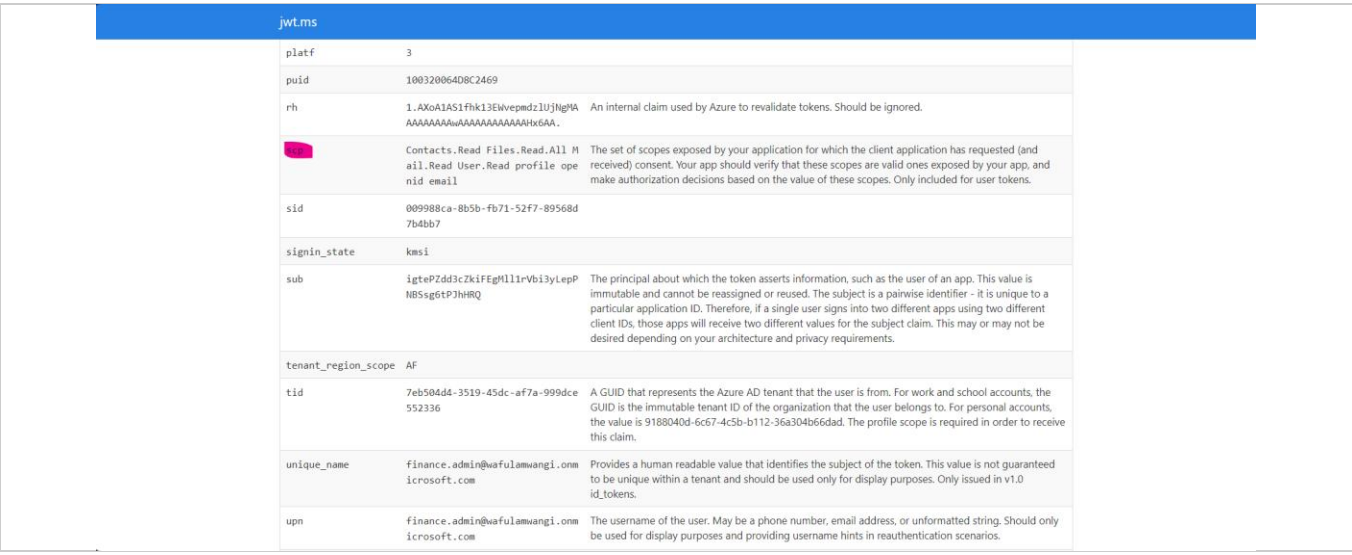


Figure 5 — Decoded token Claims tab: scp and upn claims

Key claims confirmed the attack succeeded:

- scp: Contacts.Read Files.Read.All Mail.Read User.Read profile openid email
- upn: finance.admin@wafulamwangi.onmicrosoft.com
- aud: https://graph.microsoft.com
- tid: matches the tenant's Entra ID GUID

This token, bound to the Finance Admin identity, grants the "Expense Reporting Tool" application direct read access to that user's mailbox, OneDrive/SharePoint files, and contacts — and because offline_access was also granted, a refresh token was issued, allowing the application to mint new access tokens indefinitely without requiring the user to re-authenticate.

4. Technical Method Summary

- Registered a single-tenant Entra ID app registration with a Web platform redirect URI (https://jwt.ms) for token inspection
- Added four delegated Microsoft Graph API permissions not requiring admin consent by default
- Generated a client secret for the token exchange step
- Attempted the OAuth2 authorization code flow directly against a standard user — blocked by tenant default consent policy
- Reconfigured tenant consent and permission classification settings to reflect a looser, more common real-world posture
- Used the application's /adminconsent endpoint to obtain tenant-wide admin consent
- Re-ran the standard user authorization code flow, which succeeded without further prompting
- Exchanged the resulting authorization code for an access token via PowerShell (Invoke-RestMethod against the /oauth2/v2.0/token endpoint)
- Decoded and inspected the resulting token's claims via jwt.ms

5. Impact

Once admin consent was granted, every current and future user in the tenant who signs into "Expense Reporting Tool" silently inherits read access to their mail, files, and contacts, plus a renewable refresh token enabling persistent access. With no further consent prompt, no visibility to the user that this occurred, and no requirement for the attacker-controlled application to re-request access. In a production tenant, this would represent a tenant-wide data exposure introduced by a single approval decision.

6. Detection

The following signals would be visible to a monitoring team and should drive alerting:

- Entra ID Audit Logs: "Consent to application" event with consentType = "AllPrincipals", performed by an administrator, against an application not from a verified publisher
- Entra ID Audit Logs: "Add OAuth2PermissionGrant" / "Add app role assignment grant to user" events for a newly registered or recently created enterprise application
- Sign-in Logs: subsequent delegated token issuance events for multiple distinct users against the same unverified client application shortly after the admin consent event
- Microsoft Graph Activity Logs (if enabled): API calls to /me/messages, /me/drive, /me/contacts originating from the application's client ID across multiple users
- Absence of a verified publisher domain on the application combined with broad Mail/Files/Contacts scopes and offline_access is itself a high-signal indicator worth a standing detection rule

7. Remediation Recommendations

- Set tenant user consent policy to "Do not allow user consent" and route all app approvals through a reviewed admin consent workflow
- Restrict the "low impact" permission classification list to genuinely low-risk scopes; treat offline_access and any Mail/Files/Contacts-class scope as requiring explicit review regardless of Microsoft's suggested defaults
- Require verified publisher status before admin consent is granted to any application requesting mail, file, or contact scopes
- Enable and regularly review Entra ID Access Reviews for enterprise application permission grants
- Alert on any "AllPrincipals" consent grant event in the audit log, routed to the security team for same-day review
- Periodically audit the Enterprise Applications list for unpublished or unverified apps with standing Mail/Files/Contacts/offline_access grants

8. Appendix — Evidence Index

- Figure 1 — "Need admin approval" block (Path A, direct user consent denied)
- Figure 2 — Permission classifications screen, 4 scopes added to "Low"
- Figure 3 — Admin consent screen, tenant-wide permissions request
- Figure 4 — Enterprise Applications → Permissions grid, tenant-wide grant confirmed
- Figure 5 — Decoded token Claims tab (scp / upn / aud / tid)